The Commons

A Mathematical Framework for Sovereign Resource Allocation

Abstract

We present a mathematical framework for resource allocation based on the Entity-Attribute-Value (EAV) paradigm with **observer recognition**. The system consists of entities with attributes, where special attributes create relational structure (membership), flow gradients (potentials), and allocation constraints. Each attribute has an associated observer who recognized it, creating a **distributed epistemic property attribution system**. We derive a **Constrained Weighted Allocation framework** and prove that when weights represent net-benefit gradients toward goal realization, the system converges to a Pareto-optimal equilibrium that respects both provider capacity constraints and recipient need limits. This formulation replaces rigid market clearing with a continuous optimization process that maximizes priority alignment, making honest preference signaling the unique optimal strategy.

1 The Entity-Attribute-Value Model

Definition 1 (The Commons)**.** *Let $\mathbb{E}$ be the universe of entities, $\mathbb{A}$ the universe of attributes, $\mathbb{V}$ the universe of values, $\mathbb{T}$ the universe of types, $\mathbb{O} \subseteq \mathbb{E}$ the set of observers, and $\mathbb{T}_{discrete}$ discrete time. The **Commons** is the quadruple:*

$$\mathcal{C} = \langle \mathcal{N}, \Pi, \Omega, \{\Delta_t\}_{t \in \mathbb{T}_{discrete}} \rangle$$

where:

- $\mathcal{N} \subseteq \mathbb{E}$ *is a finite set of entities*
- $\Pi : \mathcal{N} \to (\mathbb{A} \to \mathbb{V})$ *assigns each entity a mapping from attributes to values*
- $\Omega : \mathcal{N} \times \mathbb{A} \to \mathbb{O}$ *records which observer recognized each attribute*
- $\{\Delta_t\}_{t \in \mathbb{T}_{discrete}}$ *is the allocation history*
- $\mathbb{T}$ *is the universe of types, where $\mathcal{N} \subseteq \mathbb{T}$ (entities can serve as types)*

Remark 1 (Distributed Epistemic Property Attribution)**.** *The Commons uses a **distributed Entity-Attribute-Value (EAV) model with observer recognition**:*

- ***Entities*** *are the fundamental objects (nodes in traditional graph terminology)*
- ***Attributes*** *are property keys (e.g., "memberOf", "potentials", "reputation")*
- ***Values*** *are property values (sets of entities, lists of potentials, numbers, etc.)*

- ***Observers*** *are entities who recognize/assert attributes of other entities*

The observer function Ω *creates a* ***distributed epistemic system****:*

$$\Omega(e, a) = o \quad \textit{means} \quad \textit{"Observer } o \textit{ asserts that entity } e \textit{ has attribute } a\textit{"}$$

This enables:

- ***Perspective****: Different observers may recognize different attributes*
- ***Provenance****: Every attribute has a source (who recognized it)*
- ***Distribution****: No single authority on truth*
- ***Disagreement****: Multiple observers can have conflicting views*

This model is more general than traditional EAV, as it captures who knows what about whom*.*

1.1 Attribute Types

Remark 2 (Value Flexibility)**.** *Values in* $\mathbb{V}$ *can be of any type: primitives (text, numbers, booleans), structured data (lists, maps, records), or domain-specific types (potentials, entity references, etc.). The model imposes no restrictions on value types, allowing maximum flexibility for different use cases.*

Definition 2 (Attribute Recognition)**.** *An* ***attribute recognition*** *(or* ***assertion****) is a tuple:*

$$(o, e, a, v) \in \mathbb{O} \times \mathcal{N} \times \mathbb{A} \times \mathbb{V}$$

meaning "Observer o *recognizes that entity* e *has attribute* a *with value* v*".*

The system state is the set of all current recognitions:

$$\mathcal{R} = \{(o, e, a, v) \mid \Omega(e, a) = o \wedge \Pi(e)(a) = v\}$$

Remark 3 (Observer-Relative Truth)**.** *In this model, there is no objective truth about attributes — only* ***observer-relative assertions****. An entity may have different attribute values according to different observers:*

$$\Omega(e, a) = o_1 \wedge \Pi(e)(a) = v_1 \quad \textit{(according to } o_1\textit{)}$$

$$\Omega(e, a) = o_2 \wedge \Pi(e)(a) = v_2 \quad \textit{(according to } o_2\textit{)}$$

where $v_1 \neq v_2$ *represents disagreement.*

For implementation, we typically maintain a single "current view" per entity-attribute pair, with Ω *tracking the most recent or most trusted observer.*

1.2 The Membership Attribute

Definition 3 (Membership as Attribute)**.** *The* ***memberOf*** *attribute creates the relational structure. For entity $e \in \mathcal{N}$:*

$$\Pi(e)(\text{“}memberOf\text{”}) \in \mathbb{V}_{refs}$$

The membership relation E is ***emergent*** *from this attribute:*

$$E = \{(e_1, e_2) \in \mathcal{N} \times \mathcal{N} \mid e_2 \in \Pi(e_1)(\text{“}memberOf\text{”})\}$$

For $e_1, e_2 \in \mathcal{N}$, we write $e_1 \to e_2$ when $e_2 \in \Pi(e_1)(\text{“}memberOf\text{”})$, meaning “$e_1$ is a member of e_2”.

Definition 4 (Categories and Participants)**.** *For any entity $e \in \mathcal{N}$:*

$$Categories(e) = \Pi(e)(\text{“}memberOf\text{”}) \tag{1}$$

$$Participants(c) = \{e \in \mathcal{N} \mid c \in \Pi(e)(\text{“}memberOf\text{”})\} \tag{2}$$

Definition 5 (Transitive Closure)**.** *Let E^* be the reflexive transitive closure of the emergent relation E. Define:*

$$Ancestors(e) = \{a \in \mathcal{N} \mid (e, a) \in E^* \wedge e \neq a\} \tag{3}$$

$$Descendants(c) = \{d \in \mathcal{N} \mid (d, c) \in E^* \wedge d \neq c\} \tag{4}$$

These capture the full inheritance hierarchy.

Remark 4 (Graph as Emergent Structure)**.** *In the EAV model, the graph $G = \langle \mathcal{N}, E \rangle$ is not a primitive but an* ***emergent structure*** *derived from the memberOf attribute. This has profound implications:*

- *The relational structure is just another attribute, not special*
- *Entities can have arbitrary attributes beyond membership*
- *The model generalizes naturally to other relational patterns*
- *Constraints can uniformly reference any attribute*
- *Different observers may recognize different membership relations*

1.3 Observer-Based Queries

Definition 6 (Observer Query)**.** *For observer $o \in \mathbb{O}$, entity $e \in \mathcal{N}$, and attribute $a \in \mathbb{A}$, define:*

$$\Pi_o(e, a) = \begin{cases} \Pi(e)(a) & \text{if } \Omega(e, a) = o \\ \perp & \text{otherwise} \end{cases}$$

This returns the attribute value only if *observer o recognized it.*

Definition 7 (Observer Assertions)**.** *The set of all assertions by observer o is:*

$$Assertions(o) = \{(e, a, v) \mid \Omega(e, a) = o \wedge \Pi(e)(a) = v\}$$

This captures “what does observer o think about the world?”

Definition 8 (Entity Observers)**.** *The set of observers who have recognized attributes of entity e is:*

$$Observers(e) = \{o \in \mathbb{O} \mid \exists a \in \mathbb{A} : \Omega(e, a) = o\}$$

This captures "who has opinions about entity e?"

Remark 5 (Consensus and Conflict)**.** *When multiple observers have recognized the same attribute of an entity, we may need to aggregate their views. Common strategies include:*

- ***Most recent**: Use the observation with highest timestamp*
- ***Highest confidence**: Weight by observer confidence scores*
- ***Trust-weighted**: Weight by observer reputation/trust*
- ***Consensus**: Use majority value (for discrete attributes)*

For the purposes of allocation, we typically use a single "current view" per entity-attribute pair, with Ω tracking the authoritative observer for that view.

1.4 The Potentials Attribute

Definition 9 (Potential)**.** *A **potential** is a 4-tuple $p = (\tau, q, C, M)$ where:*

- *$\tau \in \mathbb{T}$ is the type (from the universe of types)*
- *$q \in \mathbb{R}^{\pm}$ is the signed magnitude*
- *$C \in \mathcal{C}$ is a constraint predicate*
- *M is metadata (a map of key-value pairs)*

*We say p is a **source** if $q > 0$ and a **sink** if $q < 0$.*

Since $\mathcal{N} \subseteq \mathbb{T}$, entities can serve as types (types-as-nodes), but types can also be abstract identifiers, strings, or other type representations not necessarily corresponding to entities in $\mathcal{N}$.

For a potential $p = (\tau, q, C, M)$, we use projection notation:

- *$\tau(p)$ denotes the type component*
- *$q(p)$ denotes the magnitude component*
- *$C(p)$ denotes the constraint component*
- *$M(p)$ denotes the metadata component*

***Metadata for Space-Time Matching**: The metadata M enables sophisticated matching beyond type compatibility. Common metadata includes:*

- ***Temporal**: time_zone, recurrence (daily/weekly/monthly/yearly), start_date, end_date, availability_window (hierarchical time patterns)*
- ***Spatial**: location_type, coordinates (latitude/longitude), city, country, online_link*

- ***Constraints***: *advance_notice_hours, booking_window_hours, mutual_agreement_required*
- ***Descriptive***: *name, unit, description*

Two potentials p_1 (source) and p_2 (sink) are ***compatible*** *if:*

1. *Type matches:* $\tau(p_1) = \tau(p_2)$
2. *Constraint satisfied:* $C(p_2)$ *accepts the source entity*
3. *Metadata compatible: A user-provided match function* $\phi(M(p_1), M(p_2))$ *returns true*

The match function ϕ can check arbitrary compatibility criteria, such as:

- *Time overlap (accounting for timezones and recurrence patterns)*
- *Location proximity or online availability*
- *Mutual agreement requirements*

Terminology: *For intuitive exposition, we refer to a potential's magnitude as its* ***capacity*** *when the potential is a source ($q > 0$) and as its* ***need*** *when the potential is a sink ($|q|$ where $q < 0$).*

Definition 10 (Potentials as Attribute)**.** *The* ***potentials*** *attribute stores flow gradients. For entity $e \in \mathcal{N}$:*

$$\Pi(e)(\text{“potentials”}) \in \mathbb{V}_{pots}$$

This is a list of potentials representing the entity's capacities (sources) and needs (sinks).

Remark 6 (Types from Universe $\mathbb{T}$)**.** *In this framework, types come from the universe $\mathbb{T}$, which includes entities as a special case ($\mathcal{N} \subseteq \mathbb{T}$). This means:*

Types can be entities *(types-as-nodes): When $\tau \in \mathcal{N}$, the type is an entity that may itself have:*

- *Categories (via memberOf attribute, enabling type hierarchies)*
- *Potentials (via potentials attribute, enabling meta-properties and flows)*
- *Participants (entities with this entity in their memberOf)*

Types can be abstract: *When $\tau \in \mathbb{T} \setminus \mathcal{N}$, the type is an abstract identifier (e.g., string, UUID) not corresponding to any entity in the system.*

This creates a flexible type system: use types-as-nodes when you want rich type metadata and hierarchies, use abstract types when you just need simple type matching. For example, an entity representing “MealType” could have “Food” and “Consumable” in its memberOf attribute, while also being referenced as the type in potentials like $(\text{MealType}, +100)$.

Definition 11 (Accessing Potentials)**.** *For entity $e \in \mathcal{N}$, we define:*

$$P(e) = \Pi(e)(\text{“potentials”})$$

This is the list of potentials for entity e. We can filter by type:

$$P(e, \tau) = \{p \in P(e) \mid \tau(p) = \tau\}$$

1.5 Constraints and Weights

Definition 12 (Constraint)**.** *A **constraint** is a function $C : \mathcal{N} \to \mathbb{R}_{\geq 0} \cup \{\infty\}$ that limits the maximum flow to each entity:*

- $C(e) = 0$ *excludes e from receiving flow*
- $C(e) = \infty$ *imposes no limit on e*
- $C(e) = k \in (0, \infty)$ *caps flow to e at k*

Constraints express policy limits, rate caps, distance bounds, and other restrictions on flow.

***Attribute-based constraints**: Constraints can reference any attribute:*

$$C_{reputation}(e) = \begin{cases} \infty & \text{if } \Pi(e)(\text{“reputation”}) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Definition 13 (Filter as Special Constraint)**.** *A **filter** is a constraint that takes only values in $\{0, \infty\}$:*

$$\Phi : \mathcal{N} \to \{0, \infty\} \subset \mathbb{R}_{\geq 0} \cup \{\infty\}$$

Filters determine binary eligibility (include/exclude) and are a special case of constraints.

Definition 14 (Weight)**.** *A **weight function** is $w : \mathcal{N} \to \mathbb{R}_{\geq 0}$ assigning preference to each entity.*

***Attribute-based weights**: Weights can reference any attribute:*

$$w_{reputation}(e) = \Pi(e)(\text{“reputation”})$$

1.6 Allocation Records

Definition 15 (Record)**.** *An **allocation record** at time t is:*

$$\Delta_t = (t, s, \tau, \{(r_i, q_i)\}_{i=1}^{n}, C, w, M)$$

where s is the source entity, τ the type, $\{(r_i, q_i)\}$ the recipient-quantity pairs, and M metadata.

2 Constrained Weighted Allocation

Remark 7 (Optimization-Based Allocation)**.** *The allocation mechanism is formulated as a **constrained optimization problem**. This departs from ad-hoc procedural rules or "min" functions. By defining a global objective function (minimizing deviation from priority-weighted targets) subject to physical constraints (needs and capacity), we derive an allocation that is mathematically guaranteed to be the "fairest" possible distribution given the competing preferences.*

2.1 Phase 1: Provider Constraints (Optimization)

Definition 16 (Provider Optimization Problem)**.** *Let a provider P have total capacity C. Let $R = \{r_1, ..., r_n\}$ be the set of eligible recipients. Let N_i be the declared need of recipient r_i. Let w_i be the provider's priority weight for recipient r_i, where $\sum w_i = 1$.*

We seek the allocation vector $A = (A_1, ..., A_n)$ that solves:

$$\textit{Minimize} \quad J(A) = \sum_{i=1}^{n} (A_i - w_i C)^2$$

Subject to constraints:

1. ***Need Constraint**: $0 \leq A_i \leq N_i \quad \forall i$*
2. ***Capacity Constraint**: $\sum_{i=1}^{n} A_i \leq C$*

Theorem 1 (Optimization Properties)**.** *This quadratic optimization problem over a convex polytope has a unique global minimum. The solution A^* represents the allocation that is "closest" (in the least-squares sense) to the provider's ideal weighted distribution $w_i C$, while strictly respecting all physical limitations.*

Remark 8 (Intutition)**.** *This formulation naturally handles scarcity and abundance:*

- ***Abundance**: If capacity is sufficient to meet all weighted targets without hitting need limits, $A_i = w_i C$.*
- ***Need Limits**: If a recipient needs less than their weighted share ($N_i < w_i C$), the solver caps them at N_i.*
- ***Surplus Redistribution**: The "unused" capacity from capped recipients is automatically redistributed to others who have unmet need, minimizing the total squared error. This replicates the "remainder redistribution" logic but as an intrinsic property of the optimization.*

2.2 Phase 2: Recipient Source Refinement

Definition 17 (Source Preference Problem)**.** *While Phase 1 optimizes from the provider's perspective, Phase 2 optimizes for the recipient.*

Let a recipient R receive a total allocation A_{total} from a set of providers $S = \{s_1, ..., s_m\}$. Let w_{recip} be the recipient's preference weights for each source ($w_{recip}(s_j)$), where $\sum w_{recip} = 1$.

Recipients seek to adjust the source mix $S = (S_1, ..., S_m)$ (where $\sum S_j = A_{total}$) to match their preferences:

$$\textit{Minimize} \quad K(S) = \sum_{j=1}^{m} (S_j - w_{recip}(s_j) A_{total})^2$$

Subject to:

1. ***Provider Limits**: S_j cannot exceed what provider s_j is willing/able to give (from Phase 1).*

2. ***Total Conservation***: $\sum S_j = A_{total}$ *(total help received remains constant).*

Remark 9 (Two-Sided Matching)**.** *The combination of Phase 1 (Provider Constraints) and Phase 2 (Recipient Refinement) creates a dynamic equilibrium that respects the autonomy of both sides.*

- *Providers set the* ***Supply Envelope*** *(maximum they will give to R).*
- *Recipients set the* ***Demand Composition*** *(which of the available providers they prioritize receiving from).*

3 Multi-Provider Coordination and Convergence

Remark 10 (From Single to Multiple Providers)**.** *The flow equations in Section 4 describe allocation from a single source to multiple recipients. In practice, multiple independent providers may simultaneously allocate to the same recipients. This section addresses how the system coordinates across providers to achieve equilibrium without centralized control.*

The key insight: each provider independently computes allocations using the Constrained Weighted Allocation (Phase 1), while recipients aggregate allocations to compute their ***deviations from source preference*** *(Phase 2). This deviation acts as a coordination signal that guides all providers toward equilibrium through iterative adjustment.*

3.1 Source Preference as Coordination Signal

Definition 18 (Distance from Ideal Source Mix)**.** *For recipient r and type τ, let S_r^τ be the set of all sources providing type τ. Define:*

Total received allocation:

$$A_r^\tau = \sum_{s \in S_r^\tau} a_{sr}^\tau$$

This aggregates allocations from all providers.

Source Deviation: *For each source s, the deviation is:*

$$D_{sr} = a_{sr} - (w_{recip}(s) \cdot A_r^\tau)$$

where $w_{recip}(s)$ is the recipient's preference for source s.

3.2 Convergence to Equilibrium

Definition 19 (Distance-Based Adjustment)**.** *The allocation adjustment based on deviation follows:*

$$a_{sr}^\tau(t+1) = a_{sr}^\tau(t) - \alpha \cdot D_{sr}(t)$$

where $\alpha \in (0, 1]$ is the damping factor.

This implements discrete-time gradient descent on the global energy landscape, minimizing both provider variance (Phase 1) and recipient composition error (Phase 2).

Remark 11 (Continuous-Time Interpretation)**.** *In the continuous-time limit (infinitesimal time steps $\Delta t \to 0$), the discrete update becomes:*

$$\frac{dE}{dt} = \lim_{\Delta t \to 0} \frac{E(t + \Delta t) - E(t)}{\Delta t} \leq -\alpha E(t)$$

This is analogous to thermodynamic systems where energy decreases monotonically toward equilibrium. However, the actual algorithm operates in discrete time with finite iteration steps.

Theorem 2 (Multi-Provider Equilibrium)**.** *In a system with multiple independent providers, the equilibrium state satisfies:*

$$\forall s, r, \tau : \quad a_{sr}^{\tau} = t_{sr}^{\tau}$$

where t_{sr}^{τ} is the target from Phase 1 (with Phase 2 refinements).

At equilibrium:

1. ***Universal satisfaction*** *(if capacity sufficient): Needs are met according to priorities.*
2. ***Full utilization*** *(if need sufficient): Capacity is exhausted.*
3. ***Optimal alignment****: All allocations match targets respecting both provider and recipient priorities.*
4. ***Pareto-optimal****: No reallocation can improve any participant without violating another's priorities.*

Proof. Each provider independently computes targets using Phase 1 and applies full utilization. The source preference adjustment ensures convergence to targets.

The system is a contraction mapping with rate $\kappa = (1 - \alpha) < 1$:

$$\|\Delta(t + 1)\|_F \leq (1 - \alpha)\|\Delta(t)\|_F$$

By the Banach fixed-point theorem, this converges to the unique fixed point. At equilibrium, $a_{sr}^{\tau} = t_{sr}^{\tau}$ for all providers. □ □

Corollary 1 (Scarcity Equilibrium)**.** *When total capacity is insufficient ($\sum_s q_s < \sum_r N_r^{\tau}$), equilibrium is characterized by:*

$$a_{sr}^{\tau} = t_{sr}^{\tau} \quad \forall s, r, \tau \quad \textit{(allocations match targets)} \tag{5}$$

$$\sum_r a_{sr}^{\tau} = q_s \quad \forall s, \tau \quad \textit{(full capacity utilization)} \tag{6}$$

$$\Delta_r^{\tau} > 0 \quad \textit{for some } r \quad \textit{(partial satisfaction)} \tag{7}$$

At this equilibrium, no provider can improve any recipient's satisfaction without violating their own priority constraints or capacity limits. The distribution of scarce resources is determined by the interplay of all provider weights $w_s(r)$ and recipient prioritizations $G_r(s)$, achieving maximal satisfaction subject to capacity constraints.

Proof. When capacity is insufficient, not all needs can be satisfied. Full utilization ensures all capacity is used ($\sum_r a^\tau_{sr} = q_s$). The distance-based adjustment converges to the state where each provider's allocation matches their target, which represents their optimal distribution given their priorities. Some recipients remain unsatisfied ($\Delta^\tau_r > 0$), but the allocation is Pareto-optimal: any reallocation that improves one recipient would require violating some provider's priority weights. □ □

Remark 12 (The Utilization-Satisfaction Trade-off)**.** *The system exhibits a fundamental trade-off between capacity utilization and need satisfaction:*

Abundance *(capacity > need):*

✓ *Need satisfaction: 100% (all recipients fully satisfied)*

× *Capacity utilization: < 100% (some capacity unused)*

• *Equilibrium:* $\Delta_r = 0$ *for all* r*, but* $\sum_r a_{sr} < q_s$

Scarcity *(capacity < need):*

× *Need satisfaction: < 100% (some recipients unsatisfied)*

✓ *Capacity utilization: 100% (all capacity allocated)*

• *Equilibrium:* $\sum_r a_{sr} = q_s$*, but* $\Delta_r > 0$ *for some* r

System priority*: The algorithm prioritizes recipient satisfaction over provider utilization. Once a recipient is satisfied ($\Delta_r = 0$), they are removed from allocation even if capacity remains. The system will not force allocation on satisfied recipients merely to achieve 100% utilization.*

3.3 Robustness to Network Partitions

Proposition 1 (Self-Correction via Negative Feedback)**.** *When network partitions create inconsistent views of total received allocation* A^τ_r*, the system self-corrects through negative feedback on the source deviation* D_{sr}*:*

Temporary over-allocation *($D_{sr} < 0$): Providers retract allocations to align with preferences.*

Temporary under-allocation *($D_{sr} > 0$): Providers expand allocations to align with preferences.*

Errors decay geometrically with rate $(1-\alpha)^t$*, ensuring rapid convergence once partitions heal.*

Proposition 2 (Bounded Oscillation)**.** *Maximum allocation is always bounded by provider capacity ($a_{sr} \leq q_s$) and recipient need ($A_r \leq N_r$). Even in worst-case network partitions where feedback is delayed, the system remains physically bounded and cannot diverge.*

3.4 Distributed Coordination Mechanisms

Remark 13 (The Distributed Challenge)**.** *In distributed systems, providers may not have consistent views of recipient states due to network delays or partitions. The framework requires a mechanism to synchronize the "Source Preference" signal without central coordination.*

Definition 20 (Recipient Broadcast Protocol)**.** *To enable coordination, recipients broadcast their current total received allocation A_r^τ to all connected providers:*

1. *Recipient computes:* $A_r^\tau = \sum_s a_{sr}^\tau$
2. *Recipient broadcasts:* $(r, \tau, A_r^\tau, timestamp)$
3. *Providers receive and cache latest* A_r^τ *value*
4. *Providers compute deviation:* $D_{sr} = a_{sr} - w_{recip}(s) \cdot A_r^\tau$

This ensures all providers act on a consistent (even if slightly stale) view of the global state.

Proposition 3 (Robustness to Stale Information)**.** *Suppose providers observe stale totals A_r^τ with bounded staleness. The system still converges because the negative feedback mechanism is robust to noise. Staleness acts as a temporary perturbation; once the network stabilizes, the system converges to the global equilibrium.*

4 Aggregation Along Edges

Definition 21 (Upward Aggregation)**.** *For entity e and type τ:*

$$Agg_\uparrow(e, \tau) = \sum_{c \in Categories(e)} \sum_{\substack{p \in P(c) \\ \tau(p) = \tau}} q(p)$$

Definition 22 (Downward Aggregation)**.** *For category c and type τ:*

$$Agg_\downarrow(c, \tau) = \sum_{e \in Participants(c)} \sum_{\substack{p \in P(e) \\ \tau(p) = \tau}} q(p)$$

Proposition 4 (Aggregation Duality)**.** *Aggregation preserves the sign structure:*

$$Agg_\uparrow(n, \tau) \in \mathbb{R}^\pm \tag{8}$$

$$Agg_\downarrow(c, \tau) \in \mathbb{R}^\pm \tag{9}$$

Proof. Both aggregations are sums of signed quantities $q(p) \in \mathbb{R}^\pm$. Sums of signed reals remain in $\mathbb{R}^\pm$. □ □

5 Emergent Properties

Theorem 3 (Entity-Type Duality)**.** *Every entity $e \in \mathcal{N}$ can simultaneously be:*

1. *A **type** if Participants$(e) \neq \emptyset$*
2. *An **instance** if Categories$(e) \neq \emptyset$ (i.e., $\Pi(e)$("memberOf") $\neq \emptyset$)*
3. *A **concrete entity** if $P(e) \neq \emptyset$ (i.e., $\Pi(e)$("potentials") $\neq \emptyset$)*
4. *A **potential type** if $\exists p \in P(e') : \tau(p) = e$ for some $e' \in \mathcal{N}$*

These roles are not mutually exclusive. Since types are entities ($\tau \in \mathcal{N}$), an entity referenced as a type in a potential can simultaneously be an instance of other types, creating rich type hierarchies.

Proof. The EAV structure makes no distinction between entities serving different roles. An entity may have:

- Other entities with it in their memberOf attribute (making it a type)
- Entities in its own memberOf attribute (making it an instance)
- Values in its potentials attribute (making it concrete)
- Be referenced as τ in other entities' potentials (making it a potential type)

All four roles can coexist. □ □

Theorem 4 (Non-Accumulation)**.** *No node can accumulate beyond its need. Formally, if r receives flow f of type τ and has need $|q_r|$ (where $q_r < 0$), then:*

$$f \leq |q_r|$$

Proof. Immediate from the Need Constraint ($0 \leq A_i \leq N_i$) in the Provider Optimization Problem. □ □

Theorem 5 (Proportional Fairness)**.** *The allocation ratios equal the weight ratios in the absence of boundary constraints. For recipients $r_1, r_2 \in R$:*

$$\frac{Flow(s, r_1, \tau)}{Flow(s, r_2, \tau)} = \frac{w(r_1)}{w(r_2)}$$

Proof. The optimization objective $\sum(A_i - w_i C)^2$ minimizes deviation from the proportional targets $w_i C$. In the absence of binding need constraints, the solution is exactly $A_i = w_i C$, satisfying proportional fairness. □ □

Theorem 6 (Determinism)**.** *The allocation function is deterministic: identical inputs produce identical outputs. Formally:*

$$\forall S_1, S_2 : \textit{if } (G_1, P_1, C_1, w_1) = (G_2, P_2, C_2, w_2) \textit{ then } Flow_1 = Flow_2$$

Proof. The Provider Optimization Problem is a quadratic optimization over a convex polytope. Such problems have a unique global minimum. Therefore, identical system states produce identical flows. □ □

6 The Three-Layer Ontology

Definition 23 (Ontological Layers)**.** *The Commons embodies three layers of reality:*

1. ***Structural Layer**: Entity-attribute structure defines what exists through relationships*
2. ***Potential Layer**: Potentials attribute defines what* could *flow through gradients*
3. ***Actual Layer**: Records $\{\Delta_t\}$ define what* did *flow through events*

Remark 14 (Ontological Priority)**.** *The layers exhibit a natural precedence:*

$$\mathcal{N}, \Pi \prec \textit{potentials} \prec \{\Delta_t\}$$

where $\prec$ denotes ontological precedence: entities and their attributes must exist before having potentials, and potentials must exist before being actualized. This is a design principle rather than a mathematical theorem.

7 Composition of Constraints and Weights

Definition 24 (Constraint Algebra)**.** *Constraints form a semilattice under minimum:*

$$(C_1 \wedge C_2)(n) = \min(C_1(n), C_2(n)) \tag{10}$$
$$\top(n) = \infty \quad \textit{(no constraint)} \tag{11}$$
$$\bot(n) = 0 \quad \textit{(total exclusion)} \tag{12}$$

The minimum operation selects the tightest constraint.

Proposition 5 (Filter Algebra as Special Case)**.** *When constraints are restricted to $\{0, \infty\}$, they form a Boolean algebra:*

$$(\Phi_1 \wedge \Phi_2)(n) = \min(\Phi_1(n), \Phi_2(n)) \tag{13}$$
$$(\Phi_1 \vee \Phi_2)(n) = \max(\Phi_1(n), \Phi_2(n)) \tag{14}$$
$$(\neg\Phi)(n) = \begin{cases} \infty & \textit{if } \Phi(n) = 0 \\ 0 & \textit{if } \Phi(n) = \infty \end{cases} \tag{15}$$

This recovers standard Boolean filter composition.

Definition 25 (Constraint Examples)**.** *Common constraint patterns include:*

$$C_{\textit{filter}}(n) = \begin{cases} \infty & \textit{if eligible}(n) \\ 0 & \textit{otherwise} \end{cases} \quad \textit{(binary)} \tag{16}$$
$$C_{\textit{cap}}(n) = k \quad \textit{(absolute limit)} \tag{17}$$
$$C_{\textit{percent}}(n) = \alpha \cdot q_s \quad \textit{(proportional limit)} \tag{18}$$
$$C_{\textit{distance}}(n) = \max(0, d_{\max} - d(s, n)) \quad \textit{(spatial decay)} \tag{19}$$
$$C_{\textit{rate}}(n, t) = r \cdot \Delta t \quad \textit{(temporal limit)} \tag{20}$$

Definition 26 (Weight Function)**.** *A **weight function** is $w : N \to \mathbb{R}_{\geq 0}$ assigning preference to each node, subject to normalization:*

$$\sum_{r \in N} w(r) = 1$$

Proposition 6 (Weight Algebra)**.** *Weights form a semiring under:*

$$(w_1 \oplus w_2)(n) = w_1(n) + w_2(n) \tag{21}$$
$$(w_1 \otimes w_2)(n) = w_1(n) \cdot w_2(n) \tag{22}$$

Note that $\otimes$ preserves proportions but requires renormalization: $w' = (w_1 \otimes w_2)/\sum_r (w_1 \otimes w_2)(r)$.

Definition 27 (Weight Examples)**.** *Common weight patterns include:*

***Uniform weights**: $w_{uniform}(n) = 1/|N|$ for all $n \in N$*

***Recognition**: Each node n assigns recognition weights $w_n : N \to [0, 1]$ with $\sum_r w_n(r) = 1$. The **mutual recognition** between v_1 and v_2 is:*

$$MR(v_1, v_2) = \min(w_{v_1}(v_2), w_{v_2}(v_1))$$

Normalized MR values into weights:

$$w_{MR}(n) = \frac{MR(p, n)}{\sum_{r \in N} MR(p, r)}$$

***Composite weights**: Combine multiple factors through multiplication and renormalization:*

$$w_{composite}(n) \propto w_1(n) \cdot w_2(n) \cdot \ldots \cdot w_k(n)$$

For example, social allocation might weight by mutual recognition and reputation:

$$w_{social}(n) = \frac{MR(provider, n) \cdot reputation(n)}{\sum_{r \in N} MR(provider, r) \cdot reputation(r)}$$

where reputation could be derived from past allocation records or other sources.

Proposition 7 (Composability)**.** *Complex allocation strategies emerge from composition:*

$$C_{complex} = C_1 \wedge C_2 \wedge \cdots \wedge C_n \quad \textit{(tightest constraint)} \tag{23}$$
$$w_{complex} \propto w_1 \otimes w_2 \otimes \cdots \otimes w_n \quad \textit{(multiplicative preference)} \tag{24}$$

8 Dynamic Equilibrium

Definition 28 (Fill Fraction)**.** *For a given allocation round, the **fill fraction** is:*

$$f = \frac{\sum_{r \in N} \sum_{\tau} Flow(s, r, \tau)}{\sum_{r \in N} \sum_{p \in P(r), q(p) < 0} |q(p)|}$$

This measures what fraction of total need is satisfied per round.

Theorem 7 (Convergence to Equilibrium)**.** *Let $\mathcal{S}^+ = \sum_{s \in N} \sum_{p \in P(s), q(p)>0} q(p)$ be total capacity and $\mathcal{S}^- = \sum_{r \in N} \sum_{p \in P(r), q(p)<0} |q(p)|$ be total need.*

Static Case: *If $\mathcal{S}^+ \geq \mathcal{S}^-$ and network state is fixed, then:*

$$\|r(t)\| \leq (1-f)^t \cdot \|r(0)\|$$

where $\|r(t)\|$ is the total unsatisfied need at time t. The system converges to full satisfaction in:

$$T_\varepsilon = O\left(\frac{\log(1/\varepsilon)}{\log(1/(1-f))}\right) \text{ rounds}$$

Dynamic Case: *When needs and capacities evolve over time, the system does not converge to a fixed point but instead recomputes optimal allocation at each time step based on the current state $S(t) = (G^{(t)}, P^{(t)}, C^{(t)}, w^{(t)})$. The allocation at time t is optimal for the state at time t, providing adaptive rather than convergent behavior.*

Proof. **Static case:** Each round satisfies fraction f of remaining needs. By the allocation capping property, receiving always reduces need. Thus:

$$\|r(t+1)\| = \|r(t) - \phi(r(t))\| \leq (1-f) \cdot \|r(t)\|$$

Iterating gives $\|r(t)\| \leq (1-f)^t \cdot \|r(0)\|$. Setting $(1-f)^t \cdot \|r(0)\| \leq \varepsilon$ and solving for t yields the convergence rate.

Dynamic case: The system recomputes optimal allocation for current state $S(t)$ at each step, maintaining instantaneous optimality without requiring convergence to zero. □ □

9 Sovereignty and Anti-Gaming

Remark 15 (Why Normalization Matters)**.** *Weight normalization ($\sum_{r \in N} w(r) = 1$) is essential for meaningful optimization. Without it, entities could assign arbitrarily large weights to all nodes, making $w(r_1)/\sum_{r'} w(r') = 1/n$ independent of preferences. Normalization creates scarcity, forcing genuine trade-offs in preference expression.*

9.1 Weak Monotonicity

Axiom 1 (Weak Monotonicity)**.** *The allocation function satisfies weak monotonicity in weights:*

$$\frac{\partial Flow(s, r, \tau)}{\partial w(r)} \geq 0$$

Increasing a recipient's weight does not decrease their allocation.

Proposition 8 (Monotonicity as Respect)**.** *Weak monotonicity is not merely technical—it is the mathematical expression of respecting sovereign preference expression.*

If violated*: Increasing weight to beneficial partner r decreases allocation to r, creating perverse incentives to misrepresent preferences.*

When satisfied*: Honest preference expression is optimal—no strategic manipulation needed.*

9.2 The Anti-Gaming Theorem

We now prove that the Commons framework guarantees anti-gaming properties when weights represent benefit gradients.

Definition 29 (Net-Benefit Gradient)**.** *For entity e with goal G_e, define the* ***net-benefit gradient*** *from provider p:*

$$\beta_e(p) = \frac{\partial \mathbb{P}(G_e)}{\partial \mathit{received}(p, e)} \in \mathbb{R}$$

This measures how much receiving from p contributes to achieving e's goal:

- $\beta_e(p) > 0$*: Receiving from p helps achieve G_e (beneficial)*
- $\beta_e(p) = 0$*: Receiving from p is neutral to G_e (irrelevant)*
- $\beta_e(p) < 0$*: Receiving from p harms achievement of G_e (costly)*

Examples*:*

- ***Hospital****: A patient receiving critical care has $\beta_{hospital}(patient) > 0$ because providing care realizes the hospital's goal of "providing critical care". The patient contributes to goal realization by being a recipient.*
- ***Need-based allocation****: An entity distributing food may set $\beta_e(p) \propto urgency(p)$ or $\beta_e(p) \propto need(p)$, prioritizing those who most need it.*
- ***Harmful relationships****: If receiving from p undermines e's goals (e.g., accepting resources with strings attached that conflict with mission), then $\beta_e(p) < 0$.*

This generalizes beyond private benefit to any *goal-directed allocation, including altruistic, mission-driven, or collective goals.*

Theorem 8 (Anti-Gaming)**.** *Suppose entity e controls weights $w_e : N \to \mathbb{R}_{\geq 0}$ subject to normalization $\sum_r w_e(r) = 1$. If:*

1. *Allocation satisfies the Constrained Weighted Allocation*
2. *Weights satisfy weak monotonicity (Axiom 1)*
3. *Entity seeks to maximize goal achievement:* $\max_{w_e} \mathbb{P}(G_e)$

Then the optimal weight allocation is:

$$w_e^*(r) \propto \max(0, \beta_e(r))$$

Setting weights proportional to positive *net-benefit gradients is optimal.*

Implication*: Honest signaling of relationship benefit is the best strategy. Entities with $\beta_e(r) \leq 0$ should receive zero weight. Gaming (allocating to non-beneficial or harmful partners) is strictly suboptimal.*

Proof. Entity e's goal achievement depends on received allocations:

$$\mathbb{P}(G_e) = f\left(\sum_{p \in N} \beta_e(p) \cdot \text{Flow}(p, e, \tau)\right)$$

From the Constrained Weighted Allocation optimization:

- The system minimizes $\sum (A_i - w_i C)^2$.
- In the unconstrained regime (not hitting need limits), the solution matches the target: $A_i = w_i C$.

Substituting this unconstrained solution:

$$\mathbb{P}(G_e) = f\left(\sum_{p \in N} \beta_e(p) \cdot q_p \cdot w_e(p)\right)$$

To maximize, take derivative with respect to $w_e(p)$ subject to $\sum_r w_e(r) = 1$:

$$\mathcal{L} = \sum_p \beta_e(p) \cdot q_p \cdot w_e(p) - \lambda\left(\sum_r w_e(r) - 1\right)$$

First-order conditions:

$$\frac{\partial \mathcal{L}}{\partial w_e(p)} = \beta_e(p) \cdot q_p - \lambda = 0$$

Therefore:

$$w_e^*(p) \propto \beta_e(p) \cdot q_p$$

Conclusion: Optimal weights are proportional to the product of net-benefit gradients and provider capacities: $w_e^*(p) \propto \max(0, \beta_e(p) \cdot q_p)$. This naturally accounts for varying provider capacities: larger capacity providers receive proportionally more weight. Allocating weight to neutral partners ($\beta_e(p) = 0$) or harmful partners ($\beta_e(p) < 0$) is strictly suboptimal regardless of their capacity. □ □

Remark 16 (Capacity-Aware Weighting Emerges Naturally)**.** *Recipients don't need explicit knowledge of provider capacities. The iterative convergence process naturally teaches them: providers with larger capacity consistently deliver more, leading recipients to increase their expectations from those providers over time. This is a form of* ***learned prioritization*** *where the system discovers optimal weights through experience, not through explicit capacity information exchange.*

Corollary 2 (Honest Signaling)**.** *Under the Commons framework:*

- ***Beneficial partners****: Allocate weight proportional to benefit → receive more*
- ***Non-beneficial partners****: Allocate zero weight → receive nothing*
- ***Gaming (false signaling)****: Allocate weight to non-beneficial partners → reduces allocation from beneficial ones → decreases goal achievement*

Honest preference expression is optimal.

Corollary 3 (Sybil Resistance)**.** *Creating fake identities (Sybils) provides no advantage:*

***Proof**: If entity e creates Sybil s and allocates weight $w_e(s) > 0$:*

- *Sybil s has zero capacity ($q_s = 0$) or provides no benefit ($\beta_e(s) = 0$)*
- *By anti-gaming theorem, $w_e^*(s) = 0$*
- *Allocating $w_e(s) > 0$ reduces weight available for beneficial partners*
- *Result: Decreases goal achievement*

Sybil creation is strictly suboptimal. □

9.3 Controllability and Memory

Proposition 9 (Controllability)**.** *The Commons framework permits memory in weight functions, provided current weights retain ultimate control.*

***Valid (memory with controllability)**:*

$$w^{(t)}(r) = \alpha \cdot w^{(t)}_{current}(r) + \beta \cdot w^{(t-1)}_{past}(r) \quad \text{with } \alpha > 0$$

Current weights can override past patterns.

***Invalid (memory without controllability)**:*

$$w^{(t)}(r) = \begin{cases} w^{(t)}_{current}(r) & \text{if never decreased in past} \\ 0 & \text{if ever decreased in past} \end{cases}$$

Past actions create dead zones—violates sovereignty.

***Key insight**: Learning, adaptation, and temporal smoothing are permitted as long as current weights can achieve any desired allocation.*

10 The Paradigm Shift

Proposition 10 (Unification)**.** *The Commons framework provides a unified treatment of:*

1. ***Categorization** (memberOf attribute creating emergent graph)*
2. ***Resource allocation** (potentials attribute enabling flow equilibration)*
3. ***Type inheritance** (upward aggregation along memberOf)*
4. ***Computation** (constraint satisfaction + preference optimization)*

Each emerges as a pattern within the Entity-Attribute-Value model.

Proof.

- Categorization: memberOf attribute creates type hierarchies (Definition 5)
- Allocation: The Constrained Weighted Allocation distributes potentials (Theorem 1)
- Inheritance: $\text{Agg}_\uparrow$ propagates along memberOf (Definition 11)
- Computation: Constraints C limit possibilities, weights w optimize preferences (Section 6)

Each is expressible within $\mathcal{C} = \langle \mathcal{N}, \Pi, \{\Delta_t\} \rangle$. □ □

11 A Simple Example

Consider a kitchen k with meal potential $(\texttt{meals}, +100)$ and two people: Alice with potential $(\texttt{meals}, -20)$ and Bob with potential $(\texttt{meals}, -30)$. Using unit weights ($w(n) = 0.5$ for all n), the Provider Optimization Problem seeks:

$$\text{Minimize } (A_{\text{Alice}} - 50)^2 + (A_{\text{Bob}} - 50)^2$$

Subject to:

$$0 \leq A_{\text{Alice}} \leq 20, \quad 0 \leq A_{\text{Bob}} \leq 30$$

Solution: The unconstrained targets are 50 and 50. Since $50 > 20$, Alice is capped at 20. Since $50 > 30$, Bob is capped at 30.

$$A_{\text{Alice}} = 20, \quad A_{\text{Bob}} = 30$$

Total allocated: 50. Remaining capacity: 50. Both recipients are fully satisfied. The allocation is fair, efficient, and complete relative to needs.

12 Conclusion

Five primitives—graphs, potentials, constraints, weights, and records—yield one framework. From this framework implies a remarkable property: when weights represent benefit gradients, honest preference signaling is the unique optimal strategy. Strategic manipulation, vote trading, coalition formation, and Sybil attacks all become strictly suboptimal.

This provides mathematical foundations for sovereign coordination: systems where participants retain full autonomy over their preferences while achieving collectively optimal resource allocation. Categorization, allocation, and type hierarchies all emerge as patterns within the same unified structure.